CS166 Project Phase 1

yliu954 and hshim023

1 Requirements

This project stores courses, assignments, and student grades in PostgreSQL. Students can check results and estimate grades. Professors can record grades and compare results. The Java/JDBC application is for Phase 2.

Teach represents a section. WorksIn, Enroll, and Assign store department memberships, enrollments, and results.

Relationships:

- Professors can work in multiple departments or none. Each department has at least its manager as a member. Each department has one manager; a professor can manage multiple departments.
- Each student belongs to one department. A department can have many students or none.
- Each section has one professor, course, and department. Each of those parents can have many sections or none. The professor must work in that department.
- A section can have many assignments or none. Each assignment belongs to one section.
- Students and sections have a many-to-many relationship through Enroll. Neither side needs an enrollment yet.
- Students and assignments have a many-to-many relationship through Assign. Each result must match an assignment and an enrollment in the same section. An enrollment or assignment can have no results yet.

The application will add, search, update, and delete records. Queries include a student's courses and assignments, current and projected grades, scores needed for a target grade, assignment statistics and distributions, students with no grades, GPA, professor results by term, and late submissions. Peer comparisons show summaries rather than other students' names.

Grades are percentages from 0 to 100. NULL or a missing result means ungraded; zero is a recorded grade. Dates can be unknown. Assignment IDs are local to a section and serve as labels. Names and course numbers can repeat. Graduation unit requirements can differ between students in the same department.

Assignment weights must total 100 before finalizing a grade. Phase 2 will check this cross-row total while locking Teach during changes/finalization. Drafts can be incomplete. What-if calculations do not overwrite recorded grades.

For graded work, current grade is `100 * SUM(grade * weight / 100) / SUM(weight)`. With no graded work it is unknown. Final grades require total weight 100 and all assignments graded. GPA uses rounded final percentages: A=4 at 90+, B=3 at 80+, C=2 at 70+, D=1 at 60+, and F=0. It is weighted by course units. All completed attempts count; there are no plus/minus grades.

2 Relational schema

`K = (pid, cid, sectionnumber, term)` is the section key. Below, K expands to those columns.

- **`professor(pid, name)`** - PK: pid.
- **`course(cid, coursenumber, unit)`** - PK: cid.
- **`department(did, name, manageby)`** - PK: did. Alternate key: name. FK: (manageby, did) references worksin(pid, did).
- **`student(sid, name, dept, unitrequirement)`** - PK: sid. FK: dept references department(did).
- **`worksin(pid, did)`** - PK: (pid, did). FKs: pid references professor(pid); did references department(did).
- **`teach(K, dept)`** - PK: K. FKs: cid references course(cid); (pid, dept) references worksin(pid, did).
- **`assignment(K, id, weight, duedate)`** - PK: (K, id). FK: K references teach(K).
- **`enroll(K, sid)`** - PK: (K, sid). FKs: K references teach(K); sid references student(sid).
- **`assign(K, id, sid, grade, turnindate)`** - PK: (K, id, sid). FKs: (K, id) references assignment(K, id); (K, sid) references enroll(K, sid).

In SQL, the student ID is sid, section term replaces the enrollment-only termEnrolled, and Assign includes assignment id. This separates repeated sections and results for different assignments. Department is determined by the section key.

IDs and section numbers are positive integers. Names and terms cannot be blank. Course units are 1-12 and individual unit requirements are 1-300. Weights and grades use NUMERIC(5,2); weights are greater than 0 and at most 100. Dates use DATE. Only duedate, grade, and turnindate allow NULL. Department name is the only additional candidate key. Referenced records cannot be deleted. The manager FK is deferred until commit so a department and its membership can be inserted in one transaction.

The functional dependencies are:

- professor: pid -> name.
- course: cid -> coursenumber, unit.
- department: did -> name, manageby; name -> did, manageby.
- student: sid -> name, dept, unitrequirement.
- teach: K -> dept.
- assignment: (K, id) -> weight, duedate.
- assign: (K, id, sid) -> grade, turnindate.
- worksin and enroll contain only their key columns.

These dependencies satisfy 3NF. Non-key attributes depend on the full key; there is no dependency such as dept -> unitrequirement. Names are stored once. Assignment details are separate from results, so changing a weight requires one update. Sections can exist before enrollment; deleting a result keeps its assignment. GPA and grade totals are calculated rather than stored. The enroll_sid_idx and worksin_did_idx indexes support student-history and department-staff lookups.

3 Testing

The scripts loaded successfully in two empty PostgreSQL 17.11 databases. Each run passed 42 checks and all eight example queries. An invalid manager membership also failed at commit and rolled back the transaction. The test script rolls back its own changes. The screenshots show psql output from a fresh run, displayed in a local browser page.

Database load and sample rows

PostgreSQL 17.11 - local database run on November 11, 2026

```
psql -v ON_ERROR_STOP=1 -d cs166_evidence -f sql/load.sql

BEGIN
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
ALTER TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE INDEX
CREATE INDEX
CREATE VIEW
COMMIT
BEGIN
INSERT 0 3
INSERT 0 3
INSERT 0 2
INSERT 0 4
INSERT 0 6
INSERT 0 5
INSERT 0 11
INSERT 0 12
INSERT 0 17
COMMIT

Row counts after loading
 table_name | rows
------------+------
 professor  |    3
 course     |    3
 department |    2
 worksin    |    4
 student    |    6
 teach      |    5
 assignment |   11
 enroll     |   12
 assign     |   17
(9 rows)
```

The checks cover duplicate keys, missing required fields, invalid numbers, foreign keys, and deletion of referenced records. They also check valid inserts/updates/deletes, repeated sections across terms, multiple assignment results per student, zero versus NULL grades, and incomplete weight totals.

Constraint checks and rejected records

PostgreSQL 17.11 - local database run on November 11, 2026

```
psql -qAt -v ON_ERROR_STOP=1 -d cs166_evidence -f sql/test_constraints.sql

PASS: all nine tables have expected seed counts
PASS: duplicate professor (SQLSTATE 23505)
PASS: score without assignment (SQLSTATE 23503)
PASS: score without matching enrollment (SQLSTATE 23503)
PASS: score linked to wrong term (SQLSTATE 23503)
PASS: duplicate assignment score (SQLSTATE 23505)
PASS: negative grade (SQLSTATE 23514)
PASS: grade above 100 (SQLSTATE 23514)
PASS: delete referenced student (SQLSTATE 23503)
PASS: delete referenced assignment (SQLSTATE 23503)
PASS: overweight scheme does not produce a final grade
PASS: valid insert and update
PASS: delete unreferenced record
PASS: failed grade updates left the original scores intact

All Phase 1 checks passed. Test changes rolled back.
```

Selected checks from the 42-check run. Full output is included in support/test-results.txt.

```
INSERT INTO professor VALUES (1, 'Duplicate');
ERROR:  duplicate key value violates unique constraint "professor_pkey"
DETAIL:  Key (pid)=(1) already exists.

UPDATE assign SET grade = 101 WHERE pid=1 AND cid=1001 AND sectionnumber=1 AND term='2026-
Fall' AND id=1 AND sid=101;
ERROR:  new row for relation "assign" violates check constraint "assign_grade_check"
DETAIL:  Failing row contains (1, 1001, 1, 2026-Fall, 1, 101, 101.00, 2026-10-15).

INSERT INTO assign VALUES (1,1001,1,'2026-Fall',1,105,80,NULL);
ERROR:  insert or update on table "assign" violates foreign key constraint
"assign_pid_cid_sectionnumber_term_sid_fkey"
DETAIL:  Key (pid, cid, sectionnumber, term, sid)=(1, 1001, 1, 2026-Fall, 105) is not present
in table "enroll".
```

Sample rows use fictional names and fixed dates and scores, entered directly in sample_data.sql. There are 3 professors, 3 courses, 2 departments, 4 memberships, 6 students, 5 sections, 11 assignments, 12 enrollments, and 17 results. They include late submissions, missing and ungraded work, tied scores, and grades of 0 and 100.

For student 101 in Fall CS166, the current grade is 86%. An average of 54% on the remaining work reaches a final grade of 70%; a remaining average of 90% projects to 88%. Students 104 and 106 have no recorded grades. The Spring CS166 final grades are 84%, 94%, and 66% for students 101, 102, and 103.

Grade calculation and students without grades

PostgreSQL 17.11 - local database run on November 11, 2026

```
SELECT sid, current_grade, ROUND(earned_percentage,2) AS earned, graded_weight, ROUND((70-
earned_percentage)*100/NULLIF(100-graded_weight,0),2) AS needed_for_70 FROM enrollment_grades
WHERE pid=1 AND cid=1001 AND sectionnumber=1 AND term='2026-Fall' ORDER BY sid;

 sid | current_grade | earned | graded_weight | needed_for_70
-----+---------------+--------+---------------+---------------
 101 |         86.00 |  43.00 |         50.00 |         54.00
 102 |        100.00 |  50.00 |         50.00 |         40.00
 103 |          0.00 |   0.00 |         20.00 |         87.50
 104 |               |   0.00 |             0 |         70.00
 106 |               |   0.00 |             0 |         70.00
(5 rows)


Students with no recorded grades in Fall CS166
 sid
-----
 104
 106
(2 rows)
```

Run from the submission folder:

```
createdb cs166_project
psql -X -v ON_ERROR_STOP=1 -d cs166_project -f load.sql
psql -X -v ON_ERROR_STOP=1 -d cs166_project -f support/test_constraints.sql
psql -X -v ON_ERROR_STOP=1 -d cs166_project -f support/queries.sql
```

Use an empty database or another name. Full output is in support/test-results.txt and support/screenshot-output.txt.